Experiment 1: Implement Multilayer Perceptron Algorithm for MNIST Handwritten Digit Classification

Theory

A Multilayer Perceptron (MLP) is a type of artificial neural network used for supervised learning tasks such as classification and prediction. It consists of an input layer, one or more hidden layers, and an output layer. Each neuron in one layer is connected to neurons in the next layer through weighted connections. The MNIST dataset contains grayscale images of handwritten digits from 0 to 9, where each image has a size of 28×28 pixels. Before training, the image pixels are normalized and flattened into one dimensional vectors. The MLP learns patterns in the input data using forward propagation and backpropagation algorithms. During forward propagation, input data passes through hidden layers where activation functions such as ReLU introduce nonlinearity. The output layer generally uses the Softmax activation function to classify digits into ten categories. Loss functions like categorical cross entropy measure prediction error, while optimization algorithms such as Adam update network weights. The trained model improves accuracy by minimizing error over multiple epochs. MLP models are widely used in pattern recognition, handwritten digit classification, and machine learning applications.

Learning Outcome

Learned to implement and train a Multilayer Perceptron model for handwritten digit classification using the MNIST dataset.

Experiment 2: Design a Neural Network for Classifying Movie Reviews (Binary Classification) using IMDB Dataset

Theory

Binary classification is a machine learning task in which data is categorized into one of two possible classes. In sentiment analysis of movie reviews, the objective is to determine whether a review expresses positive or negative sentiment. The IMDB dataset is a standard benchmark dataset containing thousands of labeled movie reviews. Before training the neural network, text data must undergo preprocessing steps such as tokenization, sequence encoding, and padding. Words are converted into numerical vectors because neural networks process only numerical data. An embedding layer is commonly used to represent words in dense vector form, capturing semantic relationships among words. The neural network architecture may contain embedding, dense, and dropout layers. Activation functions such as ReLU are used in hidden layers, while

the output layer generally uses the sigmoid activation function to produce probabilities between zero and one. Binary cross entropy is used as the loss function to measure prediction error. During training, the optimizer adjusts model weights to minimize loss and improve accuracy. This experiment demonstrates the application of neural networks in natural language processing tasks such as sentiment analysis, opinion mining, and text classification.

Learning Outcome

Learned to design and train a neural network model for binary sentiment classification using the IMDB movie review dataset.

Experiment 3: Design a Neural Network for Classifying News Wires (Multi Class Classification) using Reuters Dataset

Theory

Multiclass classification is a supervised learning technique in which data is classified into more than two categories. The Reuters dataset is widely used for text classification tasks and contains news articles categorized into multiple topics such as politics, business, sports, and technology. Neural networks can automatically learn complex patterns from textual data and accurately classify news articles into predefined categories. The preprocessing stage includes tokenization, integer encoding, and padding of text sequences. An embedding layer converts words into dense vectors that capture contextual information and semantic meaning. Hidden layers use activation functions such as ReLU to learn nonlinear relationships among features. The output layer uses the Softmax activation function to generate probability distributions across all classes. Categorical cross entropy is used as the loss function for multiclass classification problems. During training, the network parameters are updated using optimization algorithms like Adam or RMSprop. Model performance is evaluated using metrics such as accuracy and loss. This experiment demonstrates the effectiveness of deep learning techniques in natural language processing applications including topic categorization, document classification, and automated news filtering systems.

Learning Outcome

Learned to implement a neural network model for multiclass text classification using the Reuters newswire dataset.

Experiment 4: Design a Neural Network for Predicting House Prices using Boston Housing Price Dataset

Theory

House price prediction is a regression problem in machine learning where the objective is to estimate property prices based on different input features. The Boston Housing dataset contains information about houses such as crime rate, number of rooms, property tax, and accessibility to highways. Neural networks can model complex nonlinear relationships between input variables and house prices. Data preprocessing includes normalization or standardization of numerical features to improve training efficiency. The neural network generally consists of an input layer, hidden dense layers, and an output layer with a single neuron for predicting continuous values. Activation functions like ReLU are used in hidden layers to introduce nonlinearity. Since this is a regression problem, the output layer does not use classification activation functions such as sigmoid or softmax. Mean squared error is commonly used as the loss function to measure prediction accuracy. Optimizers like Adam adjust the network weights to minimize prediction error during training. Model performance is evaluated using metrics such as mean absolute error and root mean squared error. This experiment demonstrates the use of deep learning in predictive analytics and real estate price estimation applications.

Learning Outcome

Learned to build and evaluate a neural network regression model for predicting house prices using the Boston Housing dataset.

Experiment 4(b)(Beyond): Design a Neural Network for Predicting House Prices using Delhi Housing Price Dataset

Theory

Predicting housing prices using real world datasets is an important application of deep learning and predictive analytics. The Delhi Housing Price dataset contains information related to residential properties in Delhi, including features such as location, number of bedrooms, area, furnishing status, and amenities. Neural networks are capable of identifying hidden relationships among these features and estimating house prices accurately. The preprocessing stage includes handling missing values, converting categorical data into numerical form, and normalizing input features. The neural network architecture generally consists of multiple dense hidden layers with activation functions such as ReLU to capture nonlinear patterns in the data. The output layer contains a single neuron that predicts continuous house price values.

Mean squared error is commonly used as the loss function because it measures the difference between actual and predicted prices. Optimizers like Adam improve model performance by updating weights during training iterations. The trained model can assist in real estate valuation and market analysis. This experiment demonstrates how deep learning techniques can be applied to local housing datasets for accurate price prediction and decision making.

Learning Outcome

Learned to develop a neural network model for predicting real estate prices using the Delhi Housing Price dataset.

Experiment 5: Build a Convolution Neural Network for MNIST Handwritten Digit Classification

Theory

A Convolution Neural Network (CNN) is a deep learning model specially designed for image processing and computer vision tasks. CNNs are highly effective for recognizing patterns in images because they automatically extract important spatial features. The MNIST dataset contains grayscale images of handwritten digits from 0 to 9 with a size of 28×28 pixels. In CNN architecture, the convolution layer applies filters to the input image to detect features such as edges, textures, and shapes. Activation functions like ReLU introduce nonlinearity and improve learning capability. Pooling layers reduce the dimensions of feature maps and help decrease computational complexity while preserving important information. Flatten layers convert extracted features into one dimensional vectors for classification using dense layers. The output layer commonly uses the Softmax activation function to classify images into ten digit categories. Categorical cross entropy is used as the loss function, while optimizers such as Adam improve model performance through backpropagation. CNNs generally achieve higher accuracy than traditional neural networks for image classification tasks. This experiment demonstrates the use of deep learning techniques in handwritten digit recognition and image analysis applications effectively.

Learning Outcome

Learned to implement a Convolution Neural Network model for accurate handwritten digit classification using the MNIST dataset.

Experiment 6: Build a Convolution Neural Network for Simple Image (Dogs and Cats) Classification

Theory

Image classification is a computer vision task in which images are categorized into predefined classes based on their visual features. Convolution Neural Networks are widely used for image classification because they automatically learn important patterns from images. In the dogs and cats classification problem, the neural network is trained to distinguish between images of dogs and cats. The dataset usually contains labeled images that are resized and normalized before training. CNN architecture includes convolution layers, pooling layers, flatten layers, and fully connected dense layers. Convolution layers extract features such as edges, textures, and object shapes, while pooling layers reduce image dimensions and computational complexity. Activation functions such as ReLU improve learning efficiency, and dropout layers help prevent overfitting. The output layer generally uses the sigmoid activation function because the task involves binary classification. Binary cross entropy is used as the loss function to measure prediction error. During training, optimizers such as Adam update network weights to improve classification accuracy. This experiment

demonstrates how deep learning models can solve practical image recognition problems in computer vision, pattern recognition, and intelligent image processing applications successfully.

Learning Outcome

Learned to build and train a CNN model for binary image classification using dogs and cats image datasets.

Experiment 7: Use a Pre-trained Convolution Neural Network (VGG16) for Image Classification

Theory

Transfer learning is a deep learning technique in which a pre trained model is reused for solving similar machine learning tasks. VGG16 is a popular convolution neural network architecture developed for large scale image classification problems. It was trained on the ImageNet dataset containing millions of labeled images across thousands of categories. In this experiment, the VGG16 model is used as a feature extractor for image classification tasks. Since the model has already learned important image features such as edges, textures, and object patterns, training time and computational requirements are significantly reduced. The convolutional base of VGG16 is generally retained, while custom dense layers are added for the target classification problem. Input images are resized according to model requirements and preprocessed before training. Activation functions such as ReLU are used in hidden layers, and Softmax or sigmoid functions are used in the output layer depending on the classification type. Optimizers like Adam improve model performance by minimizing classification error. This experiment demonstrates the effectiveness of transfer learning in computer vision applications where limited datasets are available for training accurate and efficient image classification models.

Learning Outcome

Learned to apply transfer learning using the VGG16 pre trained CNN model for image classification tasks.

Experiment 7(b)(Beyond): Implement a CNN using TensorFlow/Keras to Classify the CIFAR-10 Image Dataset

Theory

The CIFAR 10 dataset is a widely used benchmark dataset for image classification tasks in deep learning. It contains sixty thousand color images of size 32×32 pixels divided into ten object categories such as airplanes, cars, birds, cats, and ships. Convolution Neural Networks are highly suitable for processing such

image datasets because they automatically learn hierarchical visual features. In this experiment, TensorFlow and Keras frameworks are used to implement and train the CNN model. The dataset is preprocessed by normalizing pixel values and converting labels into categorical form. The CNN architecture generally includes convolution layers for feature extraction, pooling layers for dimensionality reduction, dropout layers for regularization, and dense layers for classification. Activation functions such as ReLU improve the learning process, while the Softmax function is used in the output layer for multiclass prediction. Categorical cross entropy measures classification error during training, and optimizers like Adam update model parameters efficiently. Model performance is evaluated using accuracy and loss metrics. This experiment demonstrates practical implementation of deep learning models using TensorFlow and Keras for object recognition and computer vision applications in real world scenarios successfully.

Learning Outcome

Learned to implement and train a CNN model using TensorFlow and Keras for CIFAR 10 image classification.

Experiment 8: Implement One Hot Encoding of Words or Characters

Theory

One hot encoding is a data preprocessing technique used in machine learning and natural language processing to convert categorical data into numerical representation. In text processing tasks, words or characters are represented as binary vectors in which only one element is set to one while all other elements remain zero. Each unique word or character in the vocabulary is assigned a specific index position. One hot encoding allows neural networks to process textual information because machine learning algorithms require numerical input. The preprocessing process begins with tokenization, where text is divided into words or characters. A vocabulary is then created, and each token is mapped to a binary vector representation. Although one hot encoding is simple and easy to implement, it produces sparse vectors with high dimensionality when the vocabulary size increases. Despite this limitation, it is widely used in basic text classification, language modeling, and sequence processing tasks. Libraries such as TensorFlow, Keras, and NumPy provide built in functions for performing one hot encoding efficiently. This experiment demonstrates how textual data can be transformed into machine understandable numerical form for natural language processing applications effectively.

Learning Outcome

Learned to implement one hot encoding techniques for converting words or characters into numerical vector representations.